COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 18, 2026	**DATE SUBMITTED:**	September 23, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

ILO A

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Code

```cpp
#include <iostream>
#include <string>

using namespace std;

struct Node {
    char data;
    Node* firstChild;
    Node* nextSibling;

    Node(char val) : data(val), firstChild(nullptr), nextSibling(nullptr) {}
};
```

```cpp
void addChild(Node* parent, Node* child) {
    if (!parent->firstChild) {
        parent->firstChild = child;
    } else {
        Node* temp = parent->firstChild;
        while (temp->nextSibling) {
            temp = temp->nextSibling;
        }
        temp->nextSibling = child;
    }
}
```

```
71
72  // Task 1: Building the General Tree structure (Nodes A through Q)
73  Node* createGeneralTree() {
74      Node* nodeA = new Node('A');
75      Node* nodeB = new Node('B');
76      Node* nodeC = new Node('C');
77      Node* nodeD = new Node('D');
78      Node* nodeE = new Node('E');
79      Node* nodeF = new Node('F');
80      Node* nodeG = new Node('G');
81      Node* nodeH = new Node('H');
82      Node* nodeI = new Node('I');
83      Node* nodeJ = new Node('J');
84      Node* nodeK = new Node('K');
85      Node* nodeL = new Node('L');
86      Node* nodeM = new Node('M');
87      Node* nodeN = new Node('N');
88      Node* nodeP = new Node('P');
89      Node* nodeQ = new Node('Q');
90
91      // Root A children
92      addChild(nodeA, nodeB);
93      addChild(nodeA, nodeC);
94      addChild(nodeA, nodeD);
95      addChild(nodeA, nodeE);
96      addChild(nodeA, nodeF);
97      addChild(nodeA, nodeG);
98
99      // Subtree connection
100     addChild(nodeD, nodeH);
101
102     addChild(nodeE, nodeI);
103     addChild(nodeE, nodeJ);
104     addChild(nodeJ, nodeP);
105     addChild(nodeJ, nodeQ);
106
107     addChild(nodeF, nodeK);
108     addChild(nodeF, nodeL);
109     addChild(nodeF, nodeM);
110
111     addChild(nodeG, nodeN);
112
113     return nodeA;
114 }
```

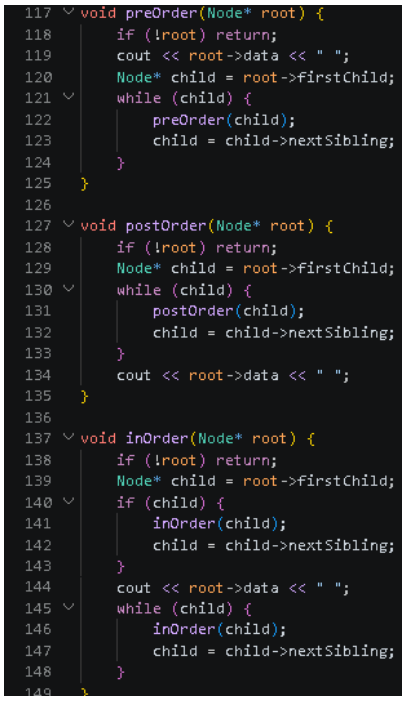
```
int main() {
    // Task 1: Create General Tree
    Node* root = createGeneralTree();
    cout << "General Tree created with Root: " << root->data << "\n\n";
```

ILO B

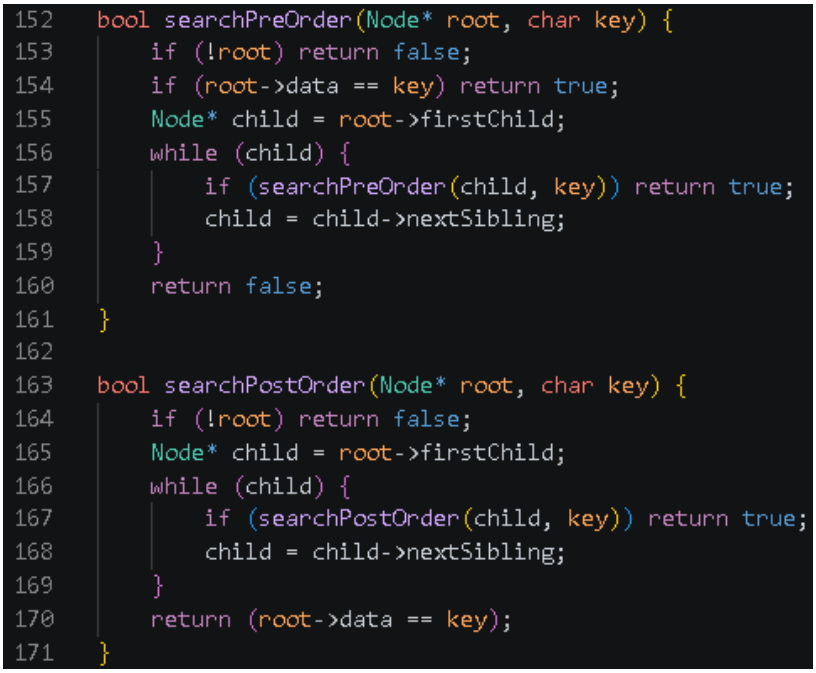
Task 3.1	
Pre-Order Traversal:	A B C D H E I J P Q F K L M G N
In-Order Traversal:	B A C H D I E P J Q K F L M N G
Post-Order Traversal:	B C H D I P Q J E K L M F N G A

Analysis:

The node depths and heights correctly match the general tree structure, where root node A has the max height of 3 and a depth of 0. Leaf nodes like B, C, and H have a height of 0 since they have no children

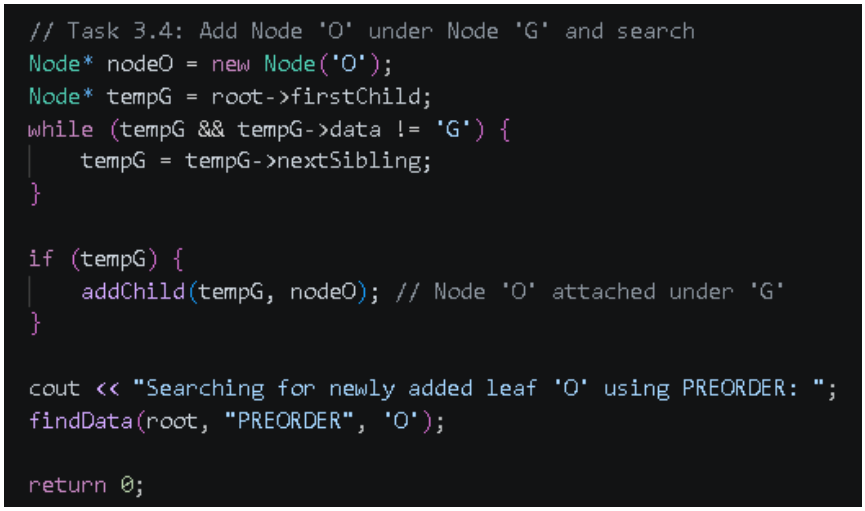
Task 3.2	
Function	
Analysis	Pre-order visits the root first before moving to child nodes, while post-order processes all children before returning to the parent. In-order traverses the leftmost child first, prints the root, and then processes the remaining sibling subtrees.

```
117 void preOrder(Node* root) {
118     if (!root) return;
119     cout << root->data << " ";
120     Node* child = root->firstChild;
121     while (child) {
122         preOrder(child);
123         child = child->nextSibling;
124     }
125 }
126
127 void postOrder(Node* root) {
128     if (!root) return;
129     Node* child = root->firstChild;
130     while (child) {
131         postOrder(child);
132         child = child->nextSibling;
133     }
134     cout << root->data << " ";
135 }
136
137 void inOrder(Node* root) {
138     if (!root) return;
139     Node* child = root->firstChild;
140     if (child) {
141         inOrder(child);
142         child = child->nextSibling;
143     }
144     cout << root->data << " ";
145     while (child) {
146         inOrder(child);
147         child = child->nextSibling;
148     }
149 }
```

Task 3.3	
Function	 
Analysis	The findData function acts as a wrapper that calls the chosen traversal algorithm based on string input. Each search function recursively checks node data and returns true as soon as a matching key is found

```
152  bool searchPreOrder(Node* root, char key) {
153      if (!root) return false;
154      if (root->data == key) return true;
155      Node* child = root->firstChild;
156      while (child) {
157          if (searchPreOrder(child, key)) return true;
158          child = child->nextSibling;
159      }
160      return false;
161  }
162
163  bool searchPostOrder(Node* root, char key) {
164      if (!root) return false;
165      Node* child = root->firstChild;
166      while (child) {
167          if (searchPostOrder(child, key)) return true;
168          child = child->nextSibling;
169      }
170      return (root->data == key);
171  }
```

```
173  bool searchInOrder(Node* root, char key) {
174      if (!root) return false;
175      Node* child = root->firstChild;
176      if (child) {
177          if (searchInOrder(child, key)) return true;
178          child = child->nextSibling;
179      }
180      if (root->data == key) return true;
181      while (child) {
182          if (searchInOrder(child, key)) return true;
183          child = child->nextSibling;
184      }
185      return false;
186  }
187
188  void findData(Node* root, string choice, char key) {
189      bool found = false;
190      if (choice == "PREORDER" || choice == "preorder") {
191          found = searchPreOrder(root, key);
192      } else if (choice == "INORDER" || choice == "inorder") {
193          found = searchInOrder(root, key);
194      } else if (choice == "POSTORDER" || choice == "postorder") {
195          found = searchPostOrder(root, key);
196      }
197
198      if (found) {
199          cout << key << " was found!" << endl;
200      }
201  }
```

Task 3.4	
Code	
Analysis	Adding node 'O' under node 'G' attaches it as a new child using sibling pointers without disrupting existing tree links. Pre-order search successfully locates 'O' by traversing down to node 'G' and checking its new child.

```
// Task 3.4: Add Node 'O' under Node 'G' and search
Node* nodeO = new Node('O');
Node* tempG = root->firstChild;
while (tempG && tempG->data != 'G') {
    tempG = tempG->nextSibling;
}

if (tempG) {
    addChild(tempG, nodeO); // Node 'O' attached under 'G'
}

cout << "Searching for newly added leaf 'O' using PREORDER: ";
findData(root, "PREORDER", 'O');

return 0;
```

PROGRAM OUTPUT

```
PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\Tree> & C:\User
ments\C++\MIDTERM\Tree\task1.exe
General Tree created with Root: A

Pre-Order Traversal: A B C D H E I J P Q F K L M G N
In-Order Traversal: B A C H D I E P J Q K F L M N G
Post-Order Traversal: B C H D I P Q J E K L M F N G A

Searching for key 'J' using INORDER: J was found!
Searching for newly added leaf 'O' using PREORDER: O was found!
PS C:\Users\mendo\OneDrive\Documents\C++\MID□ERM\Tree>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

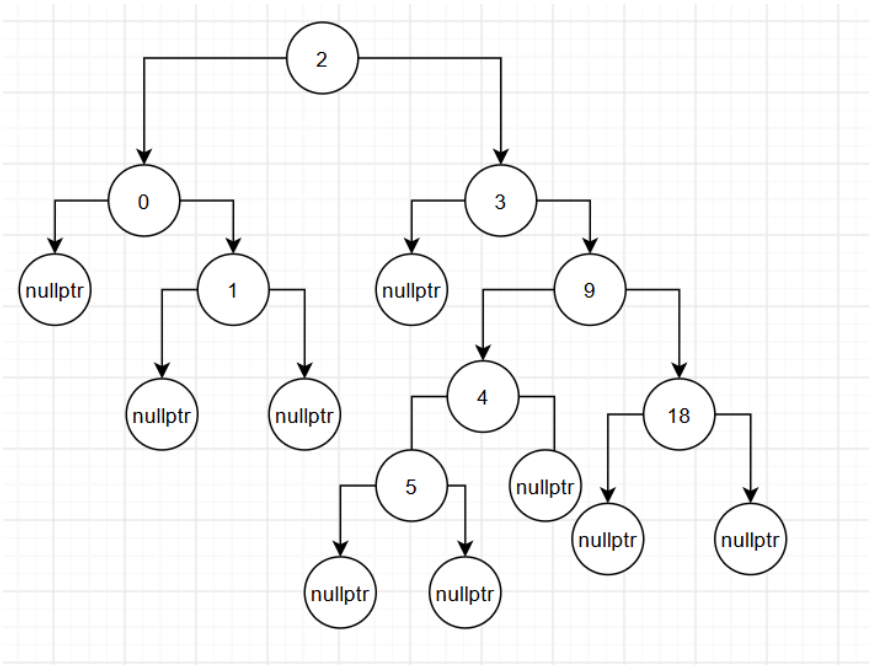
Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```
int main() {
    // Root Node
    BinarySearchTree<int> tree(2);

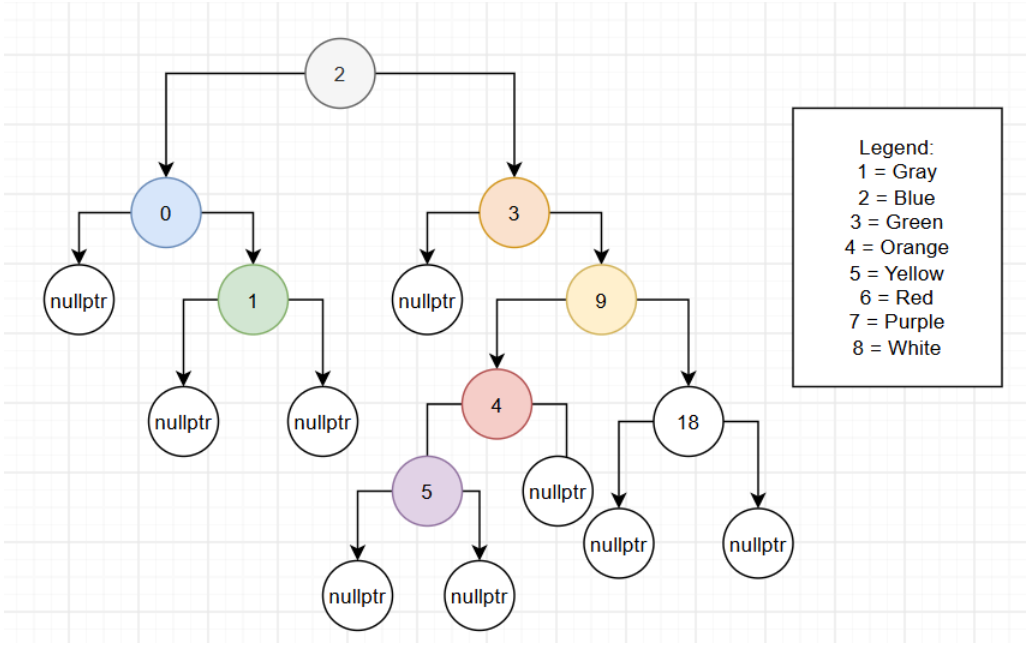
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

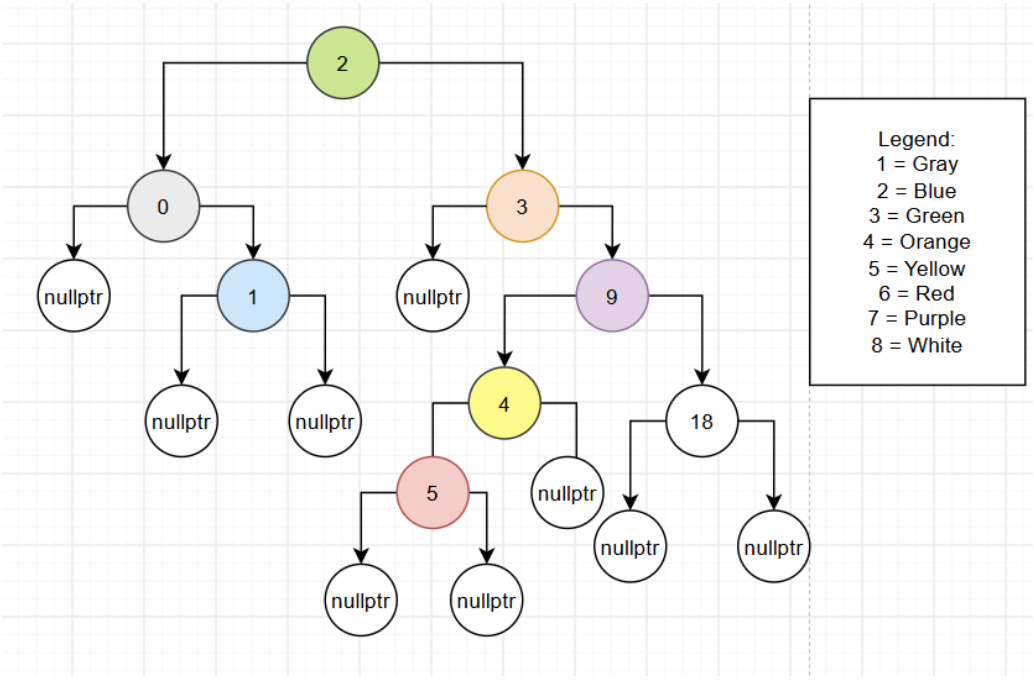
TREE DIAGRAM



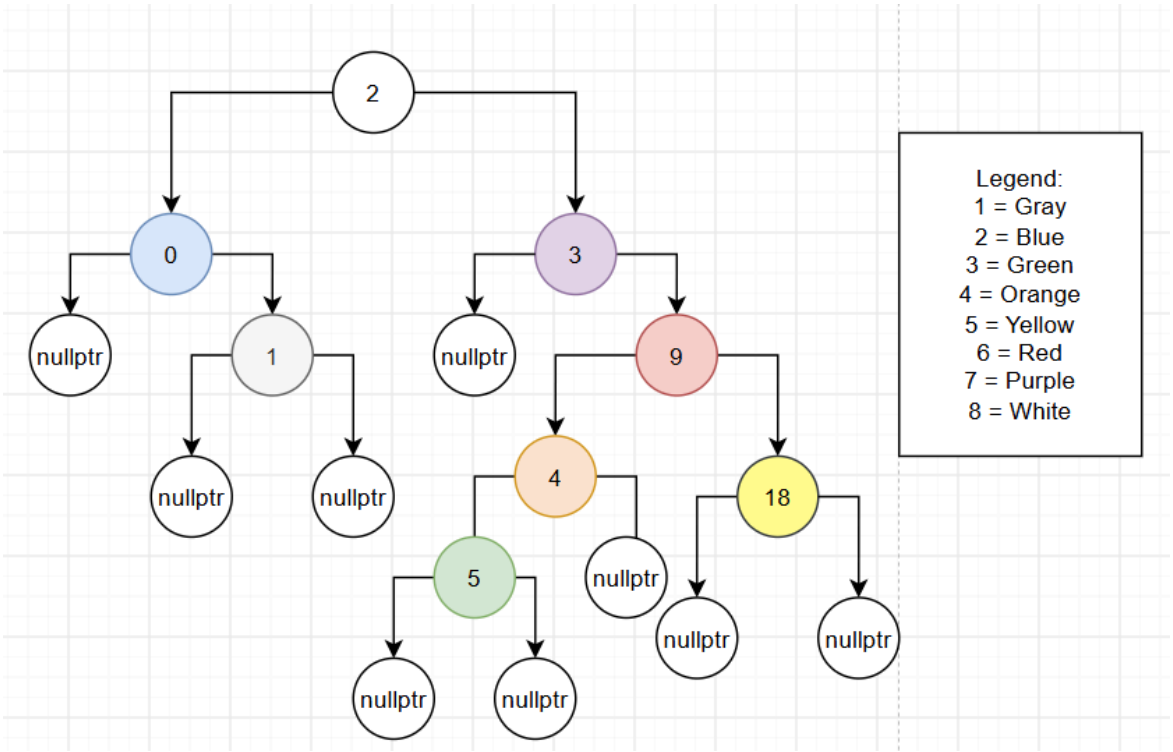
IN ORDER



PRE - ORDER



POST - ORDER


2
0
3
nullptr
1
nullptr
9
nullptr
nullptr
4
18
5
nullptr
nullptr
nullptr
nullptr
nullptr
Legend:
1 = Gray
2 = Blue
3 = Green
4 = Orange
5 = Yellow
6 = Red
7 = Purple
8 = White

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
The PreOder: 2 0 1 3 9 4 5 18
The InOrder: 0 1 2 3 4 5 9 18
The PostOrder: 1 0 5 4 18 9 3 2
```

Pre-order Traversal

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

```
The PreOder: 2 0 1 3 9 4 5 18
```

Post-order Traversal

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    // Going to the left
    if(left != nullptr){
        left->postOrder();
    }
    // Going to the right
    if(right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
```

```
The PostOrder: 1 0 5 4 18 9 3 2
```

CONCLUSION

Through this activity, general and binary trees were successfully implemented in C++ using pointer structures and different traversal algorithms. Learning pre-order, in-order, and post-order helped me understand how nodes are connected and how data can be organized in a non-linear way. I can also relate trees to real-life examples, such as family trees that show relationships between generations and biological classification that organizes living organisms from broader groups into more specific ones.

REFERENCE

https://www.geeksforgeeks.org/binary-tree-data-structure/
https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/
https://stackoverflow.com/questions/2130416/how-to-implement-a-binary-tree-in-c
https://stackoverflow.com/questions/36263596/binary-search-tree-c-template

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini / Google Search

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, understanding concepts, and asking for good/reliable references.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI and search tools were used mainly for code structural fixes, conceptual, and looking for reference.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.